

System design собес: требования и оценка нагрузки

Тебе дают открытую задачу вроде «спроектируй Twitter», и первые пятнадцать минут во многом решают исход. Интервьюер смотрит не на правильный ответ, его попросту нет, а на то, ведёшь ли ты разговор структурно: сужаешь скоуп, формулируешь требования измеримо, считаешь порядки вслух и называешь узкое место. Хаотичный поток фич без чисел проваливает собес, даже когда фичи сами по себе верные.

Скелет один и тот же на любой задаче: уточнить условие, выписать функциональные требования, перевести их в нефункциональные с числами, прикинуть нагрузку и прикинуть размер инфраструктуры. Дальше начинается архитектура, это отдельный разговор, а пока разберём первую половину: как собрать требования и прикинуть нагрузку.

Что система делает и насколько хорошо

Требования бывают двух сортов, и путать их нельзя. Функциональные говорят, что система делает для пользователя: отправить сообщение, найти ресторан поблизости, сократить ссылку. Нефункциональные говорят, насколько хорошо она это делает и при каких ограничениях: как быстро отвечает, сколько переживает отказов, во сколько обходится эксплуатация.

Быстрый тест: на идеальном компьютере (бесконечно быстром, надёжном, бесплатном) функциональное требование остаётся (сообщение надо доставить), нефункциональное исчезает. Не абсолютный (логин формально функционален), но как первое приближение работает.

Любое требование должно быть измеримым. «Система быстрая» это не требование, под него не написать тест. «Лента грузится ≤ 2 с для 90% пользователей» уже требование: есть число и способ проверить.

Главные нефункциональные измерения, по которым проходишь вслух:

Latency (задержка). Меряют перцентилем, не средним: среднее прячет хвост. p99 (99-й перцентиль) это задержка, хуже которой только 1% запросов, по ней пользователь и судит о сервисе.

Throughput (пропускная способность). RPS (requests per second); для запросов к базе QPS (queries per second).

Availability (доступность). Считают в девятках: 99.9% это ~9 часов простоя в год, 99.99% ~50 минут, 99.999% ~5 минут. Публичная гарантия это SLA (Service Level Agreement).

Важное: доступность не бесплатна, её покупают избыточностью. Чтобы пережить отказ узла, держишь копии с запасом (N+1) и раскидываешь по зонам, и чем больше девяток, тем больше избыточности: 99.9% это один регион с бэкапом, 99.99% несколько зон, 99.999% несколько регионов active-active (дорого). Цель по availability задаёт избыточность, а та умножает число серверов и бюджет, ровно как $RF \times 3$ умножает хранилище. Эту линию доводим до денег в примерах.

Durability (сохранность данных) $\neq$ availability. Availability это «отвечает ли система сейчас», durability это «не потеряются ли данные». Банк жертвует availability ради durability, лента наоборот.

Consistency (консистентность). Данные лежат в нескольких репликах, вопрос в том, что видит читатель. *Strong*: все сразу видят последнюю запись, надёжно, но дорого и медленно. *Eventual consistency*: реплики сходятся с задержкой, какое-то время читатель видит старое значение, зато быстро и дёшево. Что выбрать, подсказывает PACELC (уточнённая CAP-теорема; сама CAP гласит: при разделе сети нельзя одновременно иметь и консистентность, и доступность), читается по буквам: при разделе сети (**P**artition) выбираешь между доступностью (**A**) и консистентностью (**C**); если сеть цела (**E**lse), всё равно выбираешь между задержкой (**L**atency) и консистентностью (**C**), потому что синхронная репликация ради strong всегда тормозит. На практике ленте хватает eventual (пост дойдёт с задержкой, не страшно), банковскому балансу нужен strong.

Формулировка. Шаблон: компонент, метрика, значение, перцентиль, нагрузка. Например «поиск отвечает ≤ 200 мс для p99 в пик». Плюс бегло назови security, privacy (GDPR, персональные данные), observability (видны ли задержки и ошибки в проде), cost. Назвать их вслух уже сигнал, что думаешь шире штатного сценария.

Прикидка нагрузки на салфетке

Считают грубо, на порядки, и вслух: важен ход мысли, а не точная цифра. Идёшь по воронке от аудитории к ресурсам, на каждом шаге свой подвох.

От юзеров к RPS. DAU (daily active users) $\times$ действий в день $\div$ секунд в сутках ($\approx 100k$) = средний RPS. «Миллион записей в день» это ~ 10 RPS.

Среднее обманывает, бери пик. Нагрузка собирается в часы активности, поэтому средний RPS $\times$ peak factor (берут с потолка $\times 2-10$, у рабочих инструментов пик резче). Железо грузят на 60–70%, не под завязку.

Чтения и записи врозь. R/W обычно перекошен к чтениям, чтения уходят в кэш. 4k RPS чтения при 95% попаданий это 200 запросов до базы, не 4000. Без кэша оценка на базу завышена на порядок.

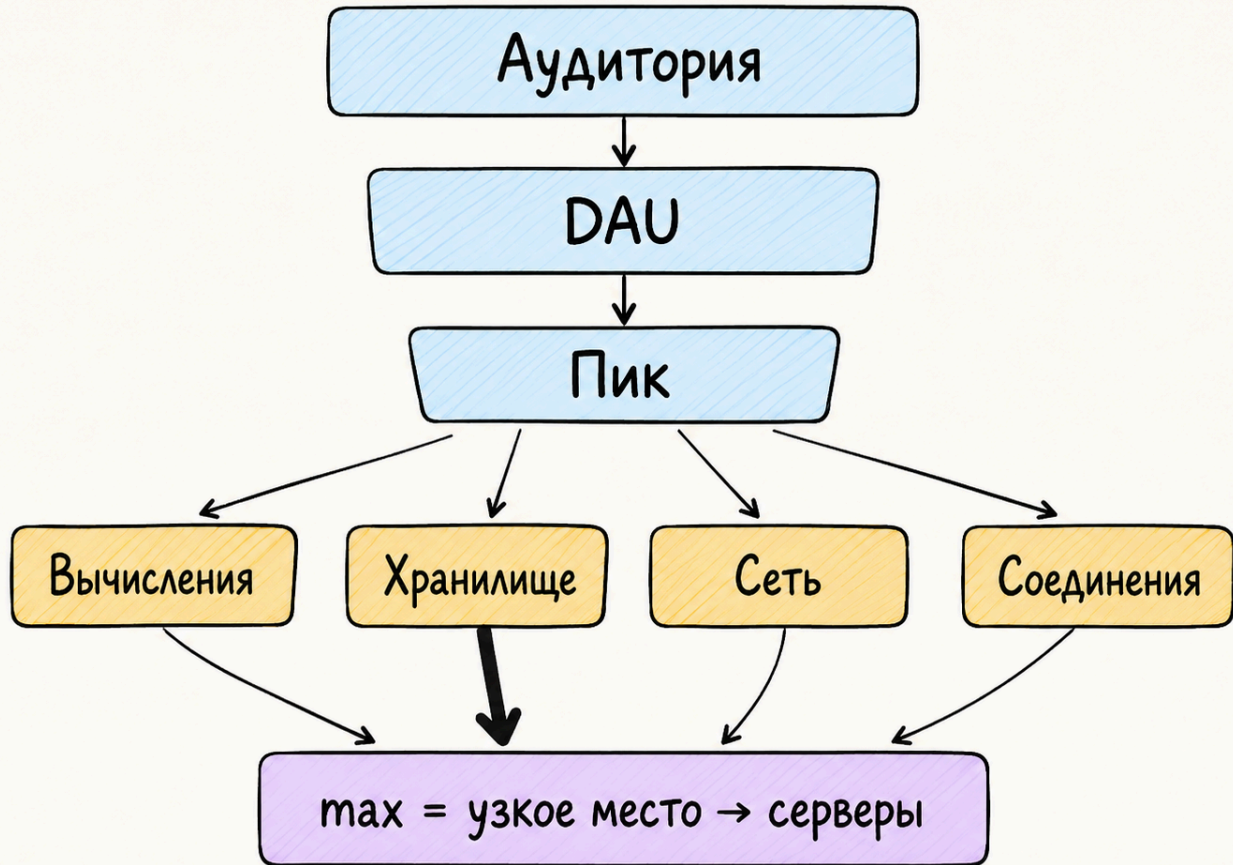
За трафик платят на отдаче. Объём = RPS × размер объекта. Загрузка в облако (ingress, входящий трафик) бесплатна, а отдача пользователю (egress, исходящий) платная, и для контентных сервисов это главная статья. Облако берёт за egress ~\$0.09/ГБ как премиум-услугу (наценка к себестоимости в десятки раз, это и заработок, и привязка клиента), поэтому отдачу уводят в CDN (content delivery network): сеть кэшей внутри сетей операторов, которая отдаёт байты ближе к пользователю и по оптовой цене ~\$0.005/ГБ, а Cloudflare R2 вообще \$0. Важная оговорка: CDN помогает только для кэшируемого (видео, картинки, статика); динамику и реалтайм (сообщения, персональные ленты, записи) он не кэширует, там платишь облачный egress либо держишь своё железо с пирингом, где байт почти даром. И различай два режима: **арендовать** CDN (Cloudflare, Fastly, Bunny, часто бесплатно) может кто угодно, это дефолт для любого, кто отдаёт статику, а не привилегия гигантов; **строить свой edge** (Netflix Open Connect, свои коробки внутри сетей операторов) нужно только на YouTube/Netflix-масштабе. На собесе: есть статика или медиа, сразу говори «ставим CDN»; свой edge обсуждай, только когда проектируешь гиганта.

Хранилище растёт только от записей. Записей в день × размер × горизонт (5 лет) × RF (replication factor, обычно 3). Этот множитель ×3 забывают: физического железа втрое больше логического, и если система упирается в объём (storage-bound), тройка прямо умножает бюджет.

Соединения это не RPS. Это две разные вещи, и их путают. Короткие запросы: сколько их в работе в один момент даёт закон Литтла (одновременных = RPS × задержка), например 60 RPS × 100 мс = 6 запросов в полёте, по этому числу подбирают размер пула воркеров. Постоянные соединения: вебсокеты или TCP-сессия (например чат, совместное редактирование) висит открытой всю сессию, поэтому их считают от числа онлайн-пользователей, а не от RPS, и у сервера на них жёсткий лимит ~1–2M (из таблицы сервера ниже).

Сколько серверов. По каждому ресурсу: пик ÷ ёмкость сервера ÷ загрузку (для хранилища спрос сперва × RF). Максимум из четырёх (вычисления, хранилище, сеть, соединения) это узкое место, его и называешь. Сверху накидываешь избыточность под availability (N+1, разные зоны). А вот latency прямого множителя к числу серверов не даёт: это требование-ограничение, оно гонит к кэшу, CDN и близким регионам и через закон Литтла задаёт размер пула воркеров, но сам парк определяют RPS, объём и соединения. Контент тяжелеет, и узкое место прыгает с вычислений на хранилище.

Оценка нагрузки: где узкое место



Воронка оценки нагрузки: аудитория сводится к пику, пик расходится на четыре ресурса, и из них побеждает максимум. Этот максимум и есть узкое место, он задаёт число серверов.

Числа, которые держишь в голове

Без них собес не сдать, с ними прикидываешь за секунды.

Перевод времени в секунды: сутки $\approx 100k$ (точно 86 400), месяц $\approx 2.5M$, год $\approx 30M$.
Делишь действия за период на эти числа и получаешь средний RPS.

Размеры типичных объектов:

Объект	Размер
символ / ASCII	1 Б
короткое сообщение	~100 Б
твит (текст)	~300 Б
запись с метаданными (JSON)	~1 КБ

Объект	Размер
миниатюра	~30–50 КБ
фото	~0.5–2 МБ
минута видео 1080p	~40 МБ
поток 1080p	~5 Мбит/с

Доступность в простое за год:

Девятки	Простой
99%	~3.7 дня
99.9%	~9 часов
99.99%	~50 минут
99.999%	~5 минут

Уровни задержки (порядок важнее точной цифры: память в тысячи раз быстрее диска, а сеть между континентами это вечность):

Операция	Задержка
обращение к RAM	~100 нс
чтение 1 МБ из RAM	~3 мкс
случайное чтение 4 КБ с SSD	~16 мкс
чтение 1 МБ с SSD	~50 мкс
round-trip (запрос туда и ответ обратно) внутри дата-центра	~500 мкс
чтение 1 МБ по сети	~1 мс
seek по HDD	~5 мс
round-trip между континентами	~150 мс

Дальше пределы и цены по трём ресурсам. Цены типичные, без учёта дефицита 2026 (сейчас SSD и RAM временно дороже в 2–5 раз).

Сервер: лимиты (что один тянет) и цена.

Параметр	Значение
app-сервер	десятки тыс. RPS на простых операциях

Параметр	Значение
БД, чтение	25–35k запросов/с на узел (масштаб через реплики; не множь на число ядер)
БД, запись	~2k надёжных записей/с (вся запись через один мастер-узел, реплики не ускоряют)
RAM на сервер	до ~1–2 ТБ
открытых соединений	1–2М на сервер (хорошо настроенный)
сетевая карта (канал в сеть)	10 / 25 / 100 Гбит/с
цена сервера	~\$5–15k (корпус без дисков-хранилища)

Хранилище и трафик: цены. Свои диски дешёвы, облако дорого; за отдачу платят больше, чем за хранение, оттого и CDN.

Что	Цена	Заметка
HDD	~\$15/ТБ разово	200–300 МБ/с, до 30 ТБ/диск, ~200 ТБ/сервер; медленный, ёмкий
SSD / NVMe	~\$50/ТБ разово	единицы ГБ/с; быстрый
RAM	~\$5/ГБ	очень быстро, стирается при выключении
облако hot (S3)	~\$20/ТБ·мес	быстрый доступ, аренда, репликация в цене
облако cold / archive	~\$2/ТБ·мес	редкий доступ, с задержкой
ingress (заливка в облако)	бесплатно	данные В облако
egress напрямую из облака	~\$0.09/ГБ (~\$90/ТБ)	дорого
egress через CDN / Cloudflare R2	~\$0.005/ГБ или \$0	дёшево, но только для кэшируемого

Примеры

Дальше пять разобранных систем, и в каждой узкое место своё: где-то кэш, где-то диск, где-то полоса, где-то соединения. На собесе тебя гоняют по разным, и полезно узнавать архетип. Notion разобрان подробно, остальные компактнее. Секунды везде по «сутки ≈ 100k».

☰ **Notion: совместный редактор, упор в хранилище и egress** >

Скоуп. Заметки с текстом и картинками, совместное редактирование, синхронизация между устройствами. Вне скоупа версии, комментарии, глобальный поиск. Читают заметно чаще, чем пишут.

Требования. Высокая доступность (99.9%, это рабочий инструмент). Совместное редактирование требует низкой задержки правок и read-your-writes для автора (свою правку он должен видеть сразу). Между устройствами хватит eventual consistency: если правка прилетит на второй девайс с задержкой в секунду, ничего страшного. Терять заметки нельзя, durability высокая.

Нагрузка. Дано: сто миллионов пользователей, миллион активных в день; каждый пишет одну заметку в день (текст плюс картинка, около 500 КБ) и читает пять. Рабочий инструмент даёт резкие пики, берём $\times 5$.

- Запись: $1\text{М в день} \div 100\text{k} \approx 10\text{ RPS}$ среднего, в пик $\times 5 \approx 50\text{ RPS}$.
- Чтение: впятеро больше, 50 RPS среднего, в пик $\approx 250\text{ RPS}$.
- Соединения: вебсокеты совместного редактирования, в пик онлайн примерно пятая часть DAU (доля прикинута с потолка), это 200k одновременных. Считается от онлайн-пользователей, а не от 50 RPS записи (закон Литтла).
- Трафик: запись (ingress, бесплатно) = $1\text{М} \times 500\text{ КБ} = 500\text{ ГБ/день}$. Чтение (egress, за него платим) = $1\text{М} \times 5 \times 500\text{ КБ} = 2.5\text{ ТБ/день}$, $\times 365 \approx \mathbf{0.9\text{ ПБ/год}}$ (в egress идёт только чтение, запись бесплатна).
- Хранилище: только записи, $500\text{ ГБ в день} \times 365 \times 5\text{ лет} \approx 0.9\text{ ПБ за 5 лет}$ (логически), с репликацией $\times 3 \approx 2.7\text{ ПБ физически}$. (Не путать с egress 0.9 ПБ в год выше: совпадение цифры случайно.)

Железо. Вычисления тут пустяк, 50 и 250 RPS база проглотит. Соединений 200k против миллиона на сервер, то есть по ёмкости хватает одного, но ради availability держим два-три в разных зонах (без этого один сервер это единая точка отказа). Узкое место это хранилище: 2.7 ПБ при 200 ТБ на сервер дают около 14 дисковых серверов. Итого парк примерно 14 дисковых плюс 3 прикладных плюс 3 под реалтайм плюс пара под кэш и балансировку, около 22 серверов. Система упирается в объём дисков (storage-bound).

Стоимость (5 лет). Три статьи: железо, сеть, и облако для сравнения. Каждая цифра с раскладкой.

- Железо (разово): $22\text{ сервера} \times \$10\text{k} = \$220\text{k}$ (корпуса без дисков) + диски на 2.7 ПБ = $2700\text{ ТБ} \times \$15/\text{ТБ} = \40k . Итого **~\$260k**.
- Сеть (egress): $0.9\text{ ПБ/год} = 900\,000\text{ ГБ}$. Напрямую из облака $\times \$0.09/\text{ГБ} = \$81\text{k} \approx \mathbf{\$80\text{k/год}}$ $\rightarrow \times 5\text{ лет} = \mathbf{\$400\text{k}}$. Через CDN $\times \$0.005/\text{ГБ} = \4.5k/год $\rightarrow \times 5 = \mathbf{\$23\text{k}}$.

- Облако вместо своих дисков (hot): храним логические 0.9 ПБ = $900 \text{ ТБ} \times \$20/\text{ТБ}/\text{мес} \times 60 \text{ мес} = \text{\$1.08M}$ за 5 лет. Это $\text{\$1.08M} / \$40k \approx 27\times$ дороже своих дисков. Вот почему объём держат на своём железе плюс CDN.

Главное. Три поправки переворачивают наивный счёт: пик $\times 5$, репликация $\times 3$ (из-за неё узкое место и есть диски) и egress как доминанта расходов. Отсюда реальная архитектура контентных сервисов: объём на своих или холодных дисках плюс CDN перед отдачей.

☰ Сокращатель ссылок (TinyURL): read-heavy, упор в кэш >

Скоуп. Превратить длинную ссылку в короткую и редиректить по короткой на исходную. Аналитика и кастомные алиасы как бонус. Чтений несоизмеримо больше записей.

Требования. Доступность критична: умер сервис, и все короткие ссылки мертвы, нужны хотя бы четыре девятки. Редирект с минимальной задержкой. Короткие коды непредсказуемы, чтобы их нельзя было подобрать.

Нагрузка. Дано: сто миллионов новых ссылок в месяц, чтений к записям сто к одному, запись около 0.5 КБ.

- Запись: $100\text{M} \div 2.5\text{M}$ секунд в месяце $\approx 40 \text{ RPS}$ среднего, трафик равномерный, в пик $\times 3 \approx 120 \text{ RPS}$.
- Чтение: в сто раз больше, около $4k \text{ RPS}$ среднего, в пик около $12k$, и почти всё отдаётся из кэша.
- Хранилище: $100\text{M} \times 12 \times 5 =$ шесть миллиардов записей по 0.5 КБ $\approx 3 \text{ ТБ}$ логически, с репликацией $\approx 9 \text{ ТБ}$. Влезает на один NVMe.

Железо и деньги. Узкого места по ёмкости нет, держим железо ради НА.

- Железо: ~ 5 серверов (2 прикладных, Redis под горячие ссылки, БД с репликой) $\times \$10k \approx \text{\$50k}$ разово; диски на 9 ТБ это $\sim \$0.5k$.
- Хранилище: 9 ТБ на одном NVMe, $\sim \$0.5k$. Ничтожно.
- Сеть: чтения это редиректы по $\sim 0.3 \text{ КБ}$. $4k \text{ RPS} \times 0.3 \text{ КБ} = 1.2 \text{ МБ/с} \times 100k = 120 \text{ ГБ/день} \times 365 \approx 44 \text{ ТБ/год} = 44\,000 \text{ ГБ} \times \$0.09/\text{ГБ} \approx \text{\$4k/год}$ (через CDN копейки).

Главное. При R/W сто к одному архитектура это кэш перед базой, ёмкости хватает с запасом. Честный случай «хватит горстки серверов», но только потому, что объёмы малы.

Скоуп. Публиковать твиты, подписываться, видеть домашнюю ленту из тех, на кого подписан. Вне скоупа поиск, тренды, реклама.

Требования. Доступность важнее консистентности: задержка появления твита в ленте на пару секунд приемлема (по CAP это AP, при разделе сети выбираем доступность, а не консистентность). Лента грузится за сотни миллисекунд для р95.

Нагрузка. Дано: двести миллионов активных в день, по два твита, лента обновляется двадцать раз в день, твит около 300 Б (медиа опускаем).

- Запись: $200M \times 2 = 400M$ в день $\div 100k \approx 4k$ RPS среднего, в пик $\times 3 \approx 12k$.
- Чтение лент: $200M \times 20 = 4B$ в день $\div 100k \approx 40k$ RPS среднего, в пик $\approx 120k$.
- Хранилище текста: $400M \times 300 \text{ Б} = 120 \text{ ГБ}$ в день, за пять лет $\approx 220 \text{ ТБ}$, с репликацией $\approx 660 \text{ ТБ}$ (медиа в реальности раздувает это до петабайт).

Развилка (fan-out, то есть рассыпка ленты). Главный вопрос это как собирать ленту. При push (рассыпка на запись) опубликованный твит сразу кладётся в ленты всех подписчиков, читать потом дёшево, но знаменитость со ста миллионами подписчиков порождает сто миллионов вставок на один твит, это горячий ключ (одна сущность, в которую бьёт непропорционально много нагрузки). При pull (сборка на чтение) писать дёшево, зато каждая загрузка ленты дорогая. На практике берут гибрид: обычным пользователям push, знаменитостям pull.

Железо и деньги.

- Железо: чтение лент $120k$ RPS из кэша плюс рассыпка это десятки прикладных и кэш-серверов, плюс ~ 3 дисковых под текст, порядок ~ 50 серверов $\times \$10k \approx \$0.5M$; диски текста $660 \text{ ТБ} \approx \$10k$.
- Хранилище: текст $660 \text{ ТБ} \times \$15/\text{ТБ} \approx \$10k$ своими дисками. Медиа (фото, видео) это петабайты, живёт в CDN, не на твоих дисках.
- Сеть: лента отдаёт ~ 20 твитов $\times 300 \text{ Б} \approx 6 \text{ КБ}$. $40k \text{ RPS} \times 6 \text{ КБ} = 240 \text{ МБ/с} \times 100k = 24 \text{ ТБ/день} \times 365 \approx 8.8 \text{ ПБ/год} = 8.8M \text{ ГБ}$. Напрямую $\times \$0.09 \approx \$790k/\text{год}$, через CDN $\times \$0.005 \approx \$44k/\text{год}$. Медиа на порядок больше и тоже через CDN, это и есть основной трафик.

Главное. Лента это предвычисление и кэш готовых лент, а не запросы к базе. Узкое место это чтения плюс размножение записей от рассыпки; гибридный fan-out для горячих ключей это сигнал зрелости.

☰ Видеостриминг (YouTube / Netflix): упор в полосу, без CDN никак >

Скоуп. Смотреть видео, искать по названию, получать рекомендации. Загрузка и транскодинг это отдельный офлайн-пайплайн, вне скоупа.

Требования. Высокая доступность, видео стартует за пару секунд и играет без буферизации. Консистентность не важна, каталог меняется редко.

Нагрузка. Дано: десять миллионов активных в день, каждый смотрит час, поток 1080p около 5 Мбит/с.

- Egress на пользователя: $5 \text{ Мбит/с} \times 3600 \text{ с} \approx 2.25 \text{ ГБ}$ в день, на всех $\approx 22 \text{ ПБ}$ в день.
- Пик по полосе: в среднем одновременно онлайн $\sim 417\text{k}$ ($10\text{М} \times \text{час} \div 24 \text{ ч}$), в час пик $\times 3 \approx 1.25\text{М}$ стримов (множитель с потолка), по 5 Мбит/с это $\sim 6 \text{ Тбит/с}$ отдачи.
- Каталог: 20k тайтлов $\times 2 \text{ ч}$ ($120 \text{ мин} \times 40 \text{ МБ} = 4.8 \text{ ГБ}$ за кодировку) $\times 10$ кодировок = $960 \text{ ТБ} \approx \mathbf{0.96 \text{ ПБ}}$, скромно на фоне трафика.

Железо и деньги.

- Железо: на origin (исходные серверы, откуда CDN берёт контент) ~ 30 серверов под раздачу и офлайн-транскодинг (перекодирование под разные качества) плюс диски под каталог, $\sim 30 \times \$10\text{k} \approx \mathbf{\$0.3\text{М}}$. Основной парк это тысячи edge-узлов CDN (близко к пользователю), их не покупаешь, а размещаешь у провайдеров.
- Хранилище: каталог $960 \text{ ТБ} \times \$15/\text{ТБ} \approx \14k своими дисками, мелочь на фоне трафика.
- Сеть (главное): $22 \text{ ПБ/день} = 22\text{М} \text{ ГБ}$. Напрямую $\times \$0.09/\text{ГБ} = \mathbf{\$2\text{М/день}}$ $\rightarrow \times 365 \approx \mathbf{\$730\text{М/год}}$ — столько платить нельзя. Через CDN/Open Connect (Netflix ставит своё железо внутри сетей операторов) отдача с edge почти даром.

Главное. Система упирается в полосу сети (bandwidth-bound): вся архитектура крутится вокруг edge-кэша, хранилище и вычисления второстепенны. Транскодинг идёт офлайн, пачками.

☰ Мессенджер (WhatsApp / Telegram): упор в соединения >

Скоуп. Личные и групповые чаты, статус онлайн и «был в сети», доставка в реальном времени. Вне скоупа звонки и каналы.

Требования. Доставка в реальном времени с минимальной задержкой. Порядок сообщений в чате консистентен. Доступность высокая. Если храним историю, сообщения не теряются.

Нагрузка. Дано: пятьсот миллионов активных в день, по пятьдесят сообщений, около 100 Б каждое.

- Запись: $500M \times 50 = 25B$ в день $\div 100k \approx 250k$ RPS среднего, в пик $\times 3 \approx 750k$.
- Доставка: каждое сообщение уходит в среднем примерно полутора получателям (личные чаты по одному, группы больше), около 375k RPS среднего.
- Соединения: постоянные TCP-сессии, в пик онлайн около двухсот миллионов. Один хорошо настроенный сервер держит около двух миллионов (классика WhatsApp на FreeBSD и Erlang), значит нужно около ста шлюзовых серверов.
- Хранилище, если хранить всё: $25B \times 100 \text{ Б} = 2.5 \text{ ТБ}$ в день, за пять лет $\approx 4.5 \text{ ПБ}$, с репликацией $\approx 13 \text{ ПБ}$ (многие удаляют после доставки).

Железо и деньги.

- Железо: ~ 100 шлюзовых (соединения, $200M \div 2M$) + ~ 65 дисковых ($13 \text{ ПБ} \div 200 \text{ ТБ}$) + ~ 35 БД/прикладных $\approx \sim 200$ серверов $\times \$10k \approx \$2M$.
- Хранилище: $13 \text{ ПБ} = 13\,000 \text{ ТБ} \times \$15/\text{ТБ} \approx \$0.2M$ своими дисками (в облаке hot были бы миллионы; многие удаляют после доставки).
- Сеть: доставка $375k/c \times 100 \text{ Б} = 37.5 \text{ МБ/с} \times 100k = 3.75 \text{ ТБ/день} \times 365 \approx 1.4 \text{ ПБ/год} = 1.4 \text{ МГБ} \times \$0.09/\text{ГБ} \approx \$120k/\text{год}$ напрямую из облака (или почти даром на своём железе с пирингом). CDN тут не помогает: сообщения динамические, не кэшируются.

Главное. Система упирается в число соединений (connection-bound): размер шлюзового слоя задаётся количеством онлайн-соединений ($\sim 2M$ на сервер), а не RPS. Самый частый случай, где мышление в RPS обманывает.

Замечания:

- Числа стоимости и отказоустойчивость дисков (AFR, annualized failure rate, порядка 1.3–1.5% в год по статистике Backblaze) меняются год к году, перепроверяй перед собесом, особенно в дефицит памяти 2026.

Источники:

- Глава «Software Requirements», SWEBOK v4 (2022)
- Google SRE Book: [SLO](#), [monitoring distributed systems](#)
- [PACELC theorem](#)
- [Latency Numbers Every Programmer Should Know](#)

- [Backblaze Drive Stats 2025](#)